



Chapter 11: Inheritance and Composition

Objectives

- In this chapter, you will:
 - Learn about inheritance
 - Learn about derived and base classes
 - Learn accessing protected members in the derived class
 - Learn about order of construction and destruction
 - Learn about dominating base class members in derived class

Objectives (cont'd.)

- Learn about overriding base class member functions in a derived class
- Learn about using derived class object as a base class object
- Learn about using derived class pointers as base class pointers
- Explore three types of inheritance: public, protected, and private
- Become familiar with the three basic principles of object-oriented design

Inheritance

- Inheritance: “is-a” relationship
 - Example: “every employee is a person”
- Inheritance allows creation of new classes from existing classes
 - Derived classes: new classes created from the existing class
 - Base class: the original class
- Derived class inherits **data members** and **member functions** of its base classes

Inheritance (cont'd.)

- Inheritance helps reduce software complexity
- Single inheritance: derived class has a single base class
- Multiple inheritance: derived class has more than one base class

Inheritance (cont'd.)

- Inheritance can be viewed as a tree-like, or hierarchical, structure between the base class and its derived classes

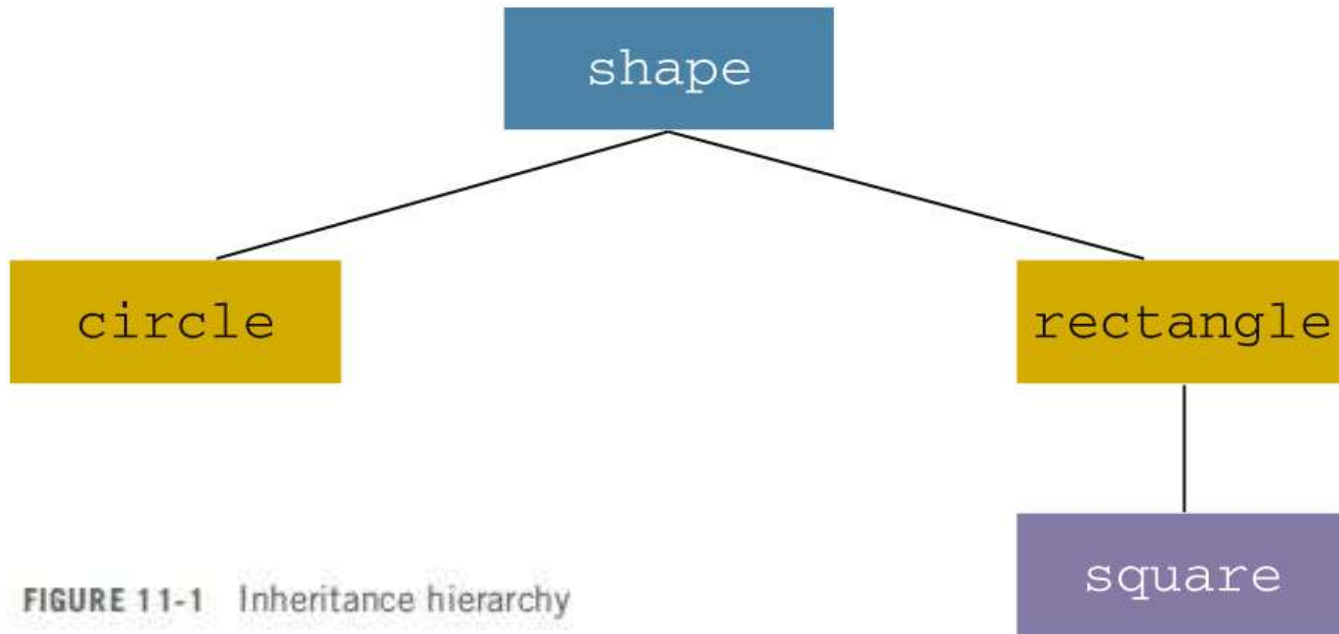


FIGURE 11-1 Inheritance hierarchy

Inheritance (cont'd.)

- Syntax of a derived class:

```
class className: memberAccessSpecifier baseClassName
{
    member list
};
```

- memberAccessSpecifier is public, protected, or private (default)
- private members of a base class are private to the base class
 - Derived class cannot directly access them

Inheritance (cont'd.)

```
#include<iostream>
using namespace std;
class Point
{
public:
    Point();
    Point(int xVal, int yVal);
    void setXY(int xVal, int yVal);
    void print();
private:
    int x;
    int y;
};

Point::Point () { x=y=0; }
Point::Point(int xVal, int yVal)
{
    setXY(xVal, yVal);
}
void Point::setXY(int xVal, int yVal)
{
    x= (xVal>=0)?xVal:0;
    y= (yVal>=0)?yVal:0;
}

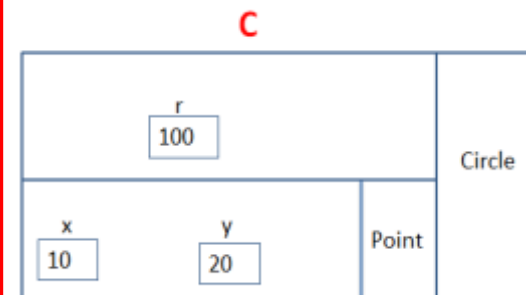
void Point::print() {cout<<'('<<x<<', '<<y<<')'<<endl;}
```

```
class Circle: public Point
{
public:
    Circle () { r=0;}
    Circle (int xVal, int yVal, int rr): Point (xVal,yVal)
    {
        r=rr;
    }
    void display()
    {
        print();
        cout<<"Radius: "<<r<<endl;
    }
private:
    int r;
};
```

```
int main()
{
    Circle c(10,20,100);
    c.display();
    return 0;
}
```

OUTPUT

(10,20)
Radius: 100



Inheritance (cont'd.)

- All member variables and functions of the base class are also member variables and functions of the derived class

Member	Access Status in Circle	How Obtained
x	Not Accessible	from class point
y	Not Accessible	from class point
print()	public	from class point
r	private	added by class circle
display()	public	added by class circle

- Derived class can include additional members (data and/or functions)

Inheritance (cont'd.)

```
int main()
{
    Circle c(10,20,100);
    c.display();
    c.setXY(30,50);
    c.print();
    return 0;
}
```

```
(10,20)
Radius: 100
(30,50)
```

OUTPUT

```
int main()
{
    Circle c(10,20,100);
    c.display();
    c.setXY(30,50);
    c.print();
    c.x=15; // ERROR
    return 0;
}
```

```
class Circle: public Point
{
public:
    Circle () { r=0;}
    Circle (int xVal, int yVal, int rr): Point (xVal,yVal)
    {
        r=rr;
    }
    void display()
    {
        print();
        cout<<'('<<x<<','<<y<<')'<<endl; //ERROR
        cout<<"Radius: "<<r<<endl;
    }
}
```

Accessing protected members in the derived class

- Derived class cannot directly access `private` members of its base class
 - To give it direct access, declare that member as `protected`

Accessing protected members in the derived class (cont'd.)

```
#include<iostream>
using namespace std;
class Point
{
public:
    Point();
    Point(int xVal, int yVal);
    void setXY(int xVal, int yVal);
    void print();
protected:
    int x;
    int y;
};

Point::Point () { x=y=0; }
Point::Point(int xVal, int yVal)
{
    setXY(xVal, yVal);
}
void Point::setXY(int xVal, int yVal)
{
    x= (xVal>=0)?xVal:0;
    y= (yVal>=0)?yVal:0;
}

void Point::print() {cout<<'('<<x<<','<<y<<')'<<endl;}
```

```
class Circle: public Point
{
public:
    Circle () { r=0;}
    Circle (int xVal, int yVal, int rr): Point (xVal,yVal)
    {
        r=rr;
    }
    void display()
    {
        //print();
        cout<<'('<<x<<','<<y<<')'<<endl;
        cout<<"Radius: "<<r<<endl;
    }

private:
    int r;
};
```

```
int main()
{
    Circle c(10,20,100);
    c.display();
    c.setXY(30,50);
    c.print();

    return 0;
}
```

OUTPUT

```
(10,20)
Radius: 100
(30,50)
```

Accessing protected members in the derived class (cont'd.)

Access Status of Circle's Members

Member	Access Status in Circle	How Obtained
x	protected	from class point
y	protected	from class point
print()	public	from class point
r	private	added by class circle
display()	public	added by class circle

Order of Construction and Destruction

- When you create an instance of a derived class the compiler calls constructors in the following orders:
 1. The constructor of the base class
 2. The constructor for any member objects (composition)
 3. The constructor of the derived class
- Destructors are called in the exact opposite order of construction order (3,2,1)

Dominating Base Class Members in Derived Class

- There are two forms for dominating:
 1. Dominating base class data members in a derived class
 - If a derived class adds a data member with the same name as a data member in the base class, the derived class **hides** the inherited data member.
 2. Dominating base class member functions in a derived class
 - A derived class can hide an inherited base class function by supplying a new version of the function with the same name but with different parameters list (overloading)

Dominating Base Class Members in Derived Class (cont'd.)

```
#include<iostream>
using namespace std;

class X
{
public:
    void h(float f) {cout<<f<<endl;}
    int a;
};

class Y: public X
{
public:
    void h(char c[]) {cout<<c<<endl;}
    void print() { cout<<a<<":"<<X::a<<endl;}
    int a;
};
```

```
int main()
{
    Y y;
    y.a=10;    // assign 10 to Y's a
    y.X::a=20; // assign 20 to X's a
    y.print();

    //=====
    y.h("Ahmed"); // calling Y's f()
    //y.h(70.7);   //ERROR:
    y.X::h(70.7); //OK
    return 0;
}
```

OUTPUT

```
10:20
Ahmed
70.7
```

Redefining (Overriding) Member Functions of the Base Class

- To redefine a `public` member function:
 - Corresponding function in derived class must have same name/number/types of parameters
- If derived class overrides a `public` member function of the base class, then to call the base class function, specify:
 - Name of the base class
 - Scope resolution operator (`::`)
 - Function name with appropriate parameter list

Redefining (Overriding) Member Functions of the Base Class (cont'd.)

```
#include<iostream>
using namespace std;

class X
{
public:
    void h(float f) {cout<<f<<endl;}
    void ov(int i) {cout<<i<<endl;}
    int a;
};

class Y: public X
{
public:
    void h(char c[]) {cout<<c<<endl;}
    void ov(int k)
    {
        cout<<k<<" : ";
        X::ov(k);
    }
    void print() { cout<<a<<":"<<X::a<<endl;}
    int a;
};
```

```
int main()
{
    Y y;
    y.ov(10);    // calling Y's ov(int)
    y.X::ov(20); // calling X's ov(int)
    return 0;
}
```

OUTPUT

```
10 : 10
20
```

A Derived Class Object as a Base Class Object

OUTPUT

Class X
Class X
Class X
Class X
Class X
Class X

```
#include<iostream>
using namespace std;

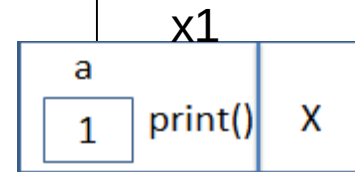
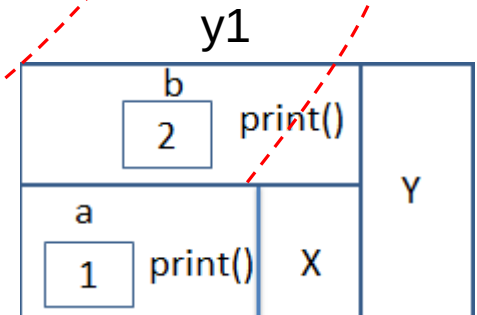
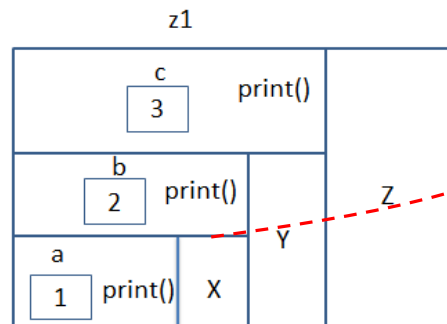
class X
{
public:
    X() { a=1; }
    void print () {cout<<"Class X \n";}
private:
    int a;
};
```

```
class Y: public X
{
public:
    Y() { b=2; }
    void print () {cout<<"Class Y \n";}
private:
    int b;
};
```

```
class Z: public Y
{
public:
    Z() { c=3; }
    void print () {cout<<"Class Z \n";}
private:
    int c;
};
```

```
int main()
{
    X x1; Y y1; Z z1;
    x1=y1; //OK: derived object is a base object
    x1.print();
    x1=z1; //OK: derived object is a base object
    x1.print();
    y1=z1; //OK: derived object is a base object
    x1.print();
    //*****
    test(x1);
    test(y1);
    test(z1);
    //*****
    //y1=x1; //ERROR: base object is not a derived object
    //z1=x1; //ERROR: base object is not a derived object
    //z1=y1; //ERROR: base object is not a derived object

    return 0;
}
```



A Derived Class Pointer as a Base Class Pointer

```
#include<iostream>
using namespace std;

class X
{
public:
    X() { a=1; }
    void print () {cout<<"Class X \n";}
private:
    int a;
};

class Y: public X
{
public:
    Y() { b=2; }
    void print () {cout<<"Class Y \n";}
private:
    int b;
};

class Z: public Y
{
public:
    Z() { c=3; }
    void print () {cout<<"Class Z \n";}
private:
    int c;
};
```

```
int main()
{

    X x; X* Px; Y y; Z z;
    Px = &x;
    Px->print();

    Px=&y;
    Px->print();
    ((Y*)Px)->print();

    Px= &z;
    Px->print();
    ((Z*)Px)->print();
    ((Y*)Px)->print();
}
```

OUTPUT

```
Class X
Class X
Class Y
Class X
Class Z
Class Y
```

C++ Stream Classes

- `ios` is the base class for all stream classes
 - Contains formatting flags and member functions to access/modify the flag settings

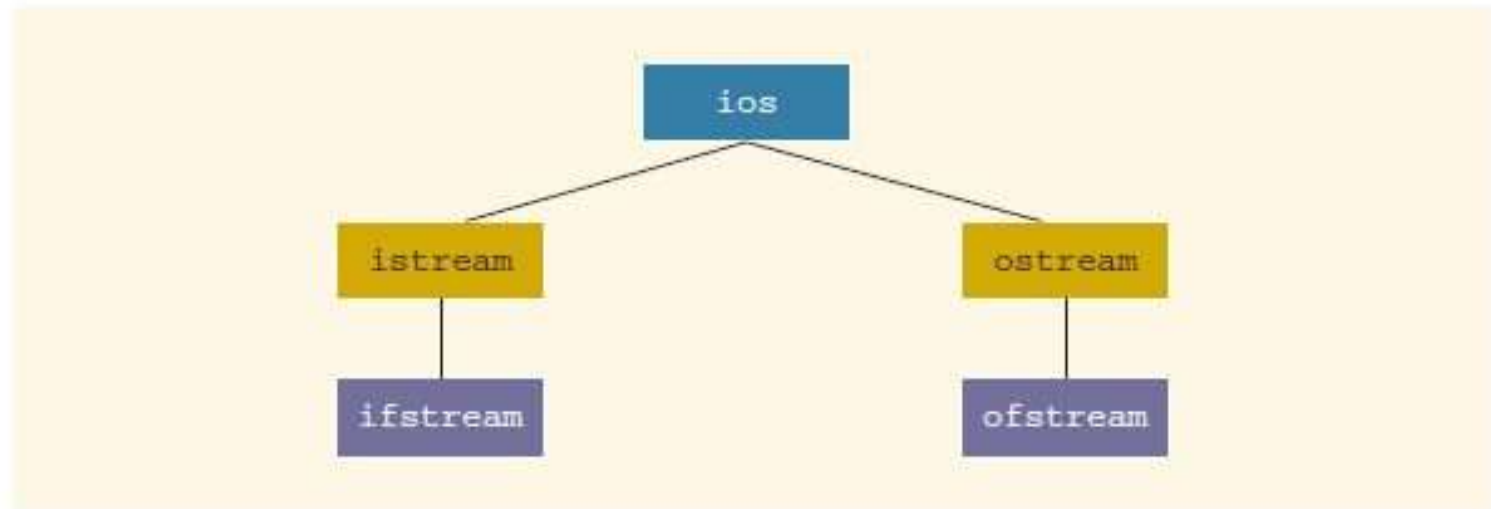


FIGURE 11-6 C++ stream classes hierarchy

C++ Stream Classes (cont'd.)

- `istream` and `ostream` provide operations for data transfer between memory and devices
 - `istream` defines the extraction operator (`>>`) and functions `get` and `ignore`
 - `ostream` defines the insertion operator (`<<`) which is used by `cout`
- `ifstream/ofstream` objects are for file I/O
 - Header file `fstream` contains the definitions for these

Inheritance as `public`, `protected`, or `private`

- Assume class B is derived from class A with

```
class B: memberAccessSpecifier A
```

- If `memberAccessSpecifier` is `public`:
 - `public` members of A are `public` in B, and can be directly accessed in class B
 - `protected` members of A are `protected` in B, and can be directly accessed by member functions (and friend functions) of B
 - `private` members of A are hidden in B and can be accessed only through `public` or `protected` members of A

Inheritance as `public`, `protected`, or `private` (cont'd.)

- If `memberAccessSpecifier` is `protected`:
 - `public` members of `A` are `protected` members of `B` and can be accessed by the member functions (and friend functions) of `B`
 - `protected` members of `A` are `protected` members of `B` and can be accessed by the member functions (and friend functions) of `B`
 - `private` members of `A` are hidden in `B` and can be accessed only through `public` or `protected` members of `A`

Inheritance as `public`, `protected`, or `private` (cont'd.)

- If `memberAccessSpecifier` is `private`:
 - `public` members of `A` are `private` members of `B` and can be accessed by member functions of `B`
 - `protected` members of `A` are `private` members of `B` and can be accessed by member functions (and friend functions) of `B`
 - `private` members of `A` are hidden in `B` and can be accessed only through `public/protected` members of `A`

Object-Oriented Design (OOD) and Object-Oriented Programming (OOP)

- The fundamental principles of object-oriented design (OOD) are:
 - Encapsulation: combines data and operations on data in a single unit
 - Inheritance: creates new objects (classes) from existing objects (classes)
 - Polymorphism: the ability to use the same expression to denote different operations

OOD and OOP (cont'd.)

- In OOD:
 - Object is a fundamental entity
 - Debug at the class level
 - A program is a collection of interacting objects
- OOD encourages code reuse
- Object-oriented programming (OOP) implements OOD

Identifying Classes, Objects, and Operations

- To find classes: begin with a problem description and identify all nouns and verbs
 - From the list of nouns choose the classes
 - From the list of verbs choose the operations
- Suppose we want to write a program that calculates and prints the volume and surface area of a cylinder

Identifying Classes, Objects, and Operations (cont'd.)

- State this problem as follows:
 - Write a **program** to *input* the **dimensions** of a **cylinder** and *calculate* and *print* the **surface area** and **volume**
 - Nouns are bold and verbs are italic
 - From the list of nouns, can visualize a cylinder as a class (`cylinderType`) from which we can create many cylinder objects of various dimensions

Identifying Classes, Objects, and Operations (cont'd.)

- These nouns are characteristics of a cylinder, so they will not be classes:
 - Dimensions
 - Surface area
 - Volume
- Next, determine three pieces of information about this class:
 - Operations that an object can perform
 - Operations that can be performed on an object
 - Information that an object must maintain

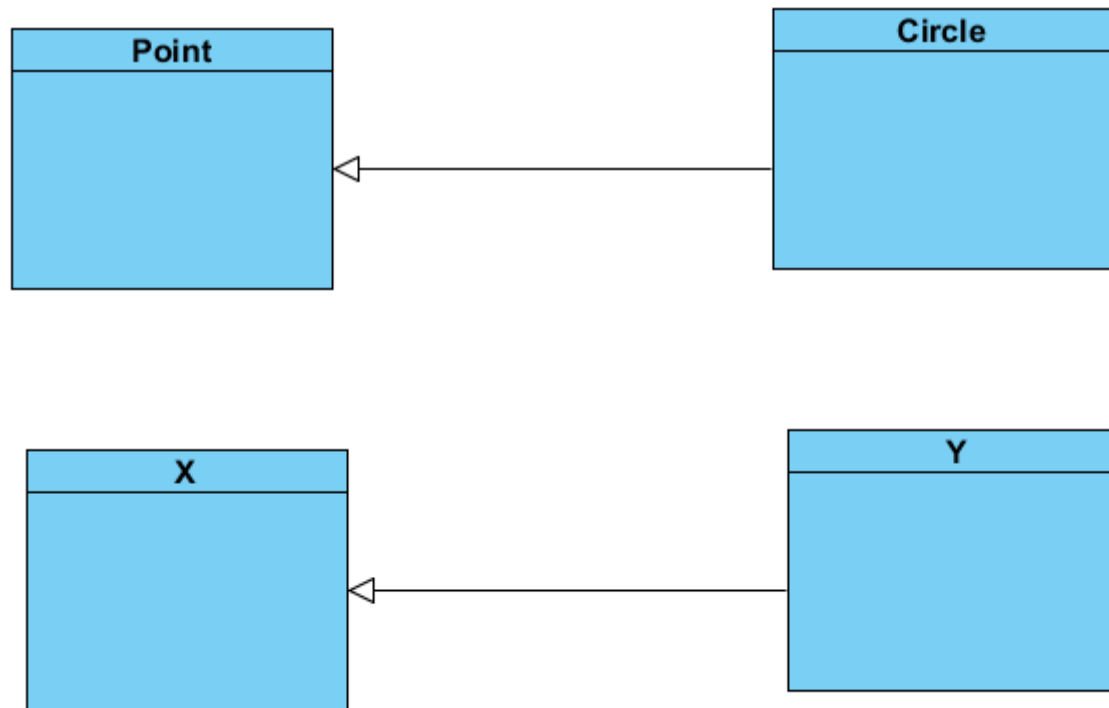
Identifying Classes, Objects, and Operations (cont'd.)

- From the verbs, list possible operations that an object of that class can perform, or have performed, on itself
 - For the `cylinderType` class:
 - Input
 - Calculate
 - Print
 - Dimensions of the cylinder represent the class's data

Identifying Classes, Objects, and Operations (cont'd.)

- Identifying classes via nouns and verbs from problem descriptions is not the only technique possible
- There are several other OOD techniques in the literature

Graphical Representation of Inheritance Relationship



Summary

- Inheritance and composition are meaningful ways to relate two or more classes
- Inheritance is an “is-a” relation
 - Single inheritance: a derived class is derived from one class, called the base class
 - Multiple inheritance: a derived class is derived from more than one base class
- Composition is a “has-a” relation

Summary (cont'd.)

- `Private` members of a base class are `private` to the base class
- `Public` members of a base class can be inherited either as `public` or `private`
- Derived class can redefine function members of a base class
 - Redefinition applies only to objects of derived class

Summary (cont'd.)

- A call to a base class constructor (with parameters) is specified in the heading of the definition of the derived class constructor
- When initializing object of a derived class, the base class constructor is executed first
- In composition (aggregation):
 - Class member is an object of another class
 - Call to constructor of member objects is specified in heading of the definition of class's constructor

Summary (cont'd.)

- Three basic principles of OOD:
 - Encapsulation
 - Inheritance
 - Polymorphism
- To find classes:
 - Describe the problem
 - Choose classes from the list of nouns
 - Choose operations from the list of verbs

- Public inheritance: all public members of base class are inherited as public members by derived class

THE END